OpenSRS - Prosty system do cyklicznego powtarzania fiszek

PK, WN

Spis Treści

1. Wstęp 3
2. Serwer HTTP - „Autumn” 3
   2.1. Podsumowanie 3
   2.2. Definiowanie endpointów 3
   2.3. Wypakowywanie danych z żądań 3
   2.4. Middleware 4
   2.5. Objekt `State` 4
3. Prosta implementacja systemu ORM 4
   3.1. Podsumowanie 4
   3.2. Definiowanie objektów 4
   3.3. API bazy danych 5
   3.4. Klasy pomocnicze 6
      3.4.1. Filtry 6
         3.4.1.1. Łączenie filtrów 6
      3.4.2. Sortowanie 6
4. Logika i endpointy systemu OpenSRS 6
   4.1. Autentykacja 6
   4.2. Lekcje 6
   4.3. Powtórki 7
5. Definiowanie nowych zestawów 7
6. Podsumowanie 7
7. Możliwe sposoby rozwoju aplikacji 7

1. Wstęp

Projekt OpenSRS ma na celu umożliwić jego użytkownikom swobodną naukę dowolnego zestawu fiszek przy użyciu aplikacji webowej hostowanej przez wbudowany serwer HTTP.

Projekt składa się z kilku warstw, z czego każda z nich zaprojektowana była ręcznie i nie jest zaimplementowana przy użyciu bibliotek zewnętrznych. Postanowiłem ustanowić takie założenia, aby lepiej zrozumieć jak gotowe narzędzia, takie jak ORM Hibernate, lub serwer Spring działają od podszewki, poprzez implementację ich uproszczonych wersji.

2. Serwer HTTP - „Autumn”

2.1. Podsumowanie

Autumn zbudowany jest w dużej mierze na bibliotece Reflections, która wyszukuje metody z określonymi adnotacjami oraz na zestawie adnotacji, które dyktują w jaki sposób każdy z endpointów działa. Definiują one, między innymi:

- Metody, które dane endpointy mogą obsłużyć
- Adres URI endpointu
- To, czy body ma być parsowane przez serwer, czy sam endpoint ma się tym zająć

Podczas uruchamiania aplikacji, Autumn buduje listę endpointów z narzuconymi priorytetami. W ten sposób, w momencie przyjścia nowego żądania, serwer jest w stanie szybko stwierdzić, który endpoint powinien być wykorzystany.

2.2. Definiowanie endpointów

Podstawowym konceptem serwera Autumn jest „endpoint”. Jest to metoda oznaczona adnotacją `@Endpoint`, która przyjmuje conajmniej jeden argument - `State` - Stan serwera, który reprezentuje zarówno żądanie przychodzące, jak i odpowiedź serwera.

```
@Endpoint("/hello") // Standardowo, handler definiowany jest dla metody GET
public void sayHello(State state) throws IOException {
    state.sendString("Hi!", "text/plain");
}
```

Dodatkowo, jeżeli endpoint należy do konkretnej grupy zdefiniowanej wewnątrz jednej klasy, klasa ta może otrzymać adnotację `@Route("/.../")`. String zawarty wewnątrz adnotacji będzie traktowany przez Autumn jako prefiks każdego z endpointów wewnątrz niej:

```
@Route("/api/")
public class TestAPI {
    @Endpoint(value = "hello", allowedMethods = { "POST" })
    public void sayHelloPost(State state) throws IOException {
        state.sendString("Hello from the POST office!", "text/plain");
    }
    @Endpoint("hello")
    public void sayHelloGet(State state) throws IOException {
        state.sendString("Hello! (GET REQUEST)", "text/plain");
    }
}
```

2.3. Wypakowywanie danych z żądań

Autumn do pobierania danych wykorzystuje adnotacje argumentów funkcji, przykładowo aby otrzymać wartość z parametru query, argument w funkcji należy oznaczyć adnotacją `@FromQuery("nazwa")`:

```
@Endpoint("hello")
public void sayHelloGet(State state, @FromQuery("test") String test) throws IOException {
    state.sendString("Hello! (GET REQUEST): " + test, "text/plain");
}
```

Podobna reguła występuje w przypadku wypakowywania danych z:

- cookie (`@FromCookie("...")`)
- body (o ile parsowanie ciała żądania jest włączone - jest to parametr `allowParsingBody` w adnotacji `@Endpoint`) (`@FromBody`)
- ścieżki dostępu (`@FromPath("nazwa")`) - Uwaga: W takiej sytuacji, `@Route` lub `@Endpoint` muszą zawierać odpowiednią informację o ścieżce - przykładowo `@Endpoint("/fields/{field}")` oraz `@FromPath("field")`.
- parametrów „extra" narzuconych przez middleware (`@FromExtra("...")`)

2.4. Middleware

Middleware to metody podobne do endpointów, jednak zamiast adnotacji `@Endpoint`, wykorzystywana jest adnotacja `@Middleware`. Mogą one posługiwać się tymi samymi metodami ekstrakcji danych, co endpointy.

Middleware dzielą się na lokalne (takie, które działają tylko wewnątrz klasy, w której są definiowane), oraz globalne (działające na wszystkie żądania).

Klasa, w której definiowany jest middleware musi mieć adnotację `@Route(...)`

Przykład:

```
@Route("/api/")
public class TestAPI {
    @Middleware(scope = Middleware.Scope.CLASS)
    public void middleware(State state) {
        state.setExtra("extra", "Hello!!");
    }
    @Endpoint("hello")
    public void sayHelloGet(State state, @FromExtra("extra") String extra)
        throws IOException
    {
        state.sendString("Hello! (GET REQUEST): " + extra, "text/plain");
    }
}
```

2.5. Objekt `State`

Objekt state, dla serwera Autumn stanowi zarówno abstrakcję żądania, jak i odpowiedzi serwera. Ma funkcje pozwalające na przykład na odczytanie zawartości strumienia przychodzącego żądania, oraz funkjce służące do ustawiania nagłówków odpowiedzi i odczytywania nagłówków przychodzących. Wszystkie metody opisane są w Javadoc'u.

3. Prosta implementacja systemu ORM

3.1. Podsumowanie

Połączenie z bazą danych reprezentuje abstrakcyjna klasa `Database`. Sam system ORM zaimplementowany został wewnątrz `SQLDatabaseImplementation` oraz objektów należących do pakietu `opensrs.dataobjects`.

System ten polega na założeniach, które spełniać musi każdy objekt, który zapisany zostanie wewnątrz bazy danych.

3.2. Definiowanie objektów

Objekty danych dziedziczą po `DataObject`, oraz implementują własną wersję enumeratora `GenericParameter`, który za zadanie ma informować jakiego typu jest dana kolumna, oraz czy dane pole

powinno zostać wprowadzone do bazy danych w przypadku operacji insert lub update. Enumerator ten stanowi także prosty w obsłudze interface, który mogą wykorzystywać implementacje bazy danych aby ustawiać i odczytywać konkretne wartości z objektów, przy pomocy funkcji `getGetter` oraz `getSetter`.

Najprostrzy typ danych (samo ID) mogłoby zostać zapisane w poniższy sposób:

```
public class SimpleDataType extends DataObject<SimpleDataType.Parameter> {
    private int id;
    public SimpleDataType() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public enum Parameter extends GenericParameter<SimpleDataType> {
        ID((BiConsumer<SimpleDataType, Integer>) Review::setId, Review::getId,
ParameterType.EQ_ONLY, false),

        private final BiConsumer<Review, ?> setter;
        private final Function<Review, ?> getter;
        private final ParameterType type;
        private final boolean insert;

        private Parameter(BiConsumer<Review, ?> setter, Function<Review, Object> getter,
ParameterType type, boolean insert) {
            this.setter = setter;
            this.getter = getter;
            this.type = type;
            this.insert = insert;
        }

        @Override
        public ParameterType getType() {
            return type;
        }

        @Override
        public BiConsumer<Review, ?> getSetter() {
            return setter;
        }

        @Override
        public Function<Review, ?> getGetter() {
            return getter;
        }

        @Override
        public boolean insert(){
            return insert;
        }
    }
}
```

3.3. API bazy danych

Implementacja klasy `Database` dla każdego typu danych musi zawierać funkcje:

```
public List<Datatype> fetchDatatypes(Filter<Datatype.Parameter> filter,
Sorter<Datatype.Parameter> sorter, int limit);
public List<Datatype> updateDatatype(Filter<Datatype.Parameter> filter,
Update<Datatype.Parameter>... updates);
```

Gdzie `DataType` to objekt, który powinien zostać zapisany w bazie.

Ponieważ system jest skonstruowany na zasanie niemutowalnego ledger'a, funkcja służąca do kasowania objektów nie powinna być zaimplementowana.

3.4. Klasy pomocnicze

3.4.1. Filtry

Filtrowanie objektów w bazie danych odbywa się przy pomocy klasy `Filter`. Klasa ta symbolizuje pojedyńcze kryterum, które każdy z objektów zwróconych przez implementację bazy danych ma spełniać. Aby wykonywać bardziej skomplikowane filtrowanie, należy wykorzystać opisaną niżej klasę `FilterChain`.

Przykład filtrowania przy pomocy pojedyńczego filtru:

```
Review r = database.fetchReview(new Filter<>(Review.Parameter.UID, Filters.EqOnly.EQUAL,
userId), null);
```

Takie zapytanie znajdzie pierwszy `Review`, którego `UID` ustawiony jest na wartość `userId`. Identyfikatory objektów mają zdefiniowany typ `EqOnly` pomimo tego, że są liczbami, z uwagi na to, iż filtrowanie po ID większych lub mniejszych niż konkretna wartość nie ma logicznego sensu.

3.4.1.1. Łączenie filtrów

Aby filtrorwać przy użyciu wielu kryterów, należy wykorzystać klasę `FilterChain`, która symbolizuje logiczną koniunkcję lub alternatywę z kilku parametrów. Warto zauważyć, że `FilterChain` implementuje `Filter`, więc może być on wykorzystywany rekurencyjnie.

Przykładowe zapytanie:

```
Review rev = database.fetchReview(new FilterChain<>(FilterChain.Junction.AND,
Arrays.asList(
    new Filter<>(Review.Parameter.UID, Filters.EqOnly.EQUAL, uid),
    new Filter<>(Review.Parameter.SET_CODE, Filters.EqOnly.EQUAL, set.hashCode()))
), null);
```

3.4.2. Sortowanie

Sortowanie zaimplementowane jest w podobny sposób do filtrów. Jedyną różnicą jest to, że w obecnej wersji tego systemu nie da się reprezentować sortowania po kilku parametrach.

Przykładowe zapytanie:

```
Review last = database.fetchReview(
    new FilterChain<>(FilterChain.Junction.AND, Arrays.asList(
        new Filter<>(Review.Parameter.UID, Filters.EqOnly.EQUAL, uid),
        new Filter<>(Review.Parameter.SET_CODE, Filters.EqOnly.EQUAL, set.hashCode()),
        new Filter(Review.Parameter.UNLOCK_DATE, Filters.Number.EGREATER, 0),
        new Filter(Review.Parameter.TYPE, Filters.EqOnly.EQUAL, Review.TYPE_REVIEW))),
    new Sorter(Review.Parameter.UNLOCK_DATE, Sorters.NumberSorter.DESCENDING));
```

4. Logika i endpointy systemu OpenSRS

4.1. Autentykacja

4.2. Lekcje

Dany Set ma w sobie wiele fiszek. Najpierw użytkownik musi przerobić daną fiszkę w ramach lekcji - nauczyć się jej aby móc zacząc ją powtarzać jako review. Niektóre fiszki mogą wymagać wiedzy z poprzednich fiszek np. kanji może składać z kilku radykałów (składowych, prostrzych znaków). Dlatego system wspiera zablokowanie lekcji za innymi lekcjami (użytkownik nie ma dostępu do lekcji o danym kanji dopóki nie przerobi jego wszsytkich radykałów). Po przerobieniu danej fiszki jako lekcja staje się ona dostępna dla użytkownika jako powrtórka, która będzie omówiona w następnej sekcji.

OpenSRS definiuje różne endpointy do obsługi lekcji zdefiniowne w pliku `Lesson.java`:
- '/api/{set}/lesson/start' - Ustawia nowe cookie sesji lekcji, które będzie użyte do grupowania elementów poznanych w tej sesji (jeśli użytkownik pozna więcej niż jedną fiszkę podczas jednego posiedzenia będą pogrupowane tym cookie).
- '/api/{set}/lesson/queue' - Daje dostęp do kolejki dostępnych dla użytkownika lekcji. Same UI setu definiuje jak zaprezentować użytkownikowi tą kolejkę.
- '/api/{set}/lesson/completed' - Pozwala przesłać informację o zakończeniu jednej lub wielu lekcji z informacją podczas jakiej sesji dana lekcja została zakończona (można przesyłać tą informajcę np. po każdej lekcji albo po wszystkich - zależy to od samego UI setu).
- '/api/{set}/lesson/lastSession' - Daje informacje zwrotne o ostatnio zakończonej sesji - może zostać wykorzystny do wyświetlenia np. ekranu podsumowania co dany użytkownik nauczył się podczas danej sesji.

4.3. Powtórki

Po tym jak użytkownik przerobi daną fiszkę jako lekcja staje się ona dostępna jako powtórka. Każdy set definiuje fazy znajomości danej fiszki - najpierw użytkownik zaczyna od najniższej fazy (np. begginer). Każde udane powtórzenie danej fiszki zwiększa poziom o 1 (numeryczna wartość reprezentująca znajomośc danej fiszki). Każda faza ma swoje zdefiniowane widełki (np. set może zdefiniować drugą fazę, intermidiate od poziomu > 3 - użytkownik osiągnie fazę intermidiate po 3 udanych powtórzeniach fiszki). Każde nieudane powtórzenie fiszki obniża poziom. Każdy set definuje dla każdej fazy, jaka jest w niej utrata poziomu (np. złe powtórzenie fiszki, w której użytkownik ma fazę Begginer może obniżyc poziom o 1, natomiast złe powtórzenie fiszki na fazie Guru może obniżyć poziom o 3) Fiszek nie powtarza się też kiedy chcę - każda powtórka ustawia czas, za który najwcześniej użytkownik powinien powtórzyć daną fiszkę. Im wyższa faza danego użytkownika, tym dłuższe okresy między kiedy należy zrobić następną powtórkę (np. powtórzenie fiszki na poziomie Begginer może ustawić powtórkę na dostepną za godzinę a powótrzenie fiszki na poziomie Guru na za dwa tygodnie).

Powtórki mogą zawierać też funkcjonalność incentyw. Każda fiszka może mieć jakieś incentywy - odpowiedzi bliskie lub związane z poprawną, które nie są jednak wymaganą odpowiedzią. Wtedy zamiast liczyć się jako nieudana próba odpowiedzenia użytkownik otrzyma podpowiedź związaną z daną incentywą.

OpenSRS definiuje różne endpointy do obsługi powtórek zdefiniowne w pliku `Lesson.java`:
- '/api/{set}/review/start' - Ustawia nowe cookie sesji powtórek, które będzie użyte do grupowania elementów powtórzonych w tej sesji (jeśli użytkownik powtórzy więcej niż jedną fiszkę podczas jednego posiedzenia będą pogrupowane tym cookie).
- '/api/{set}/review/queue' - Daje dostęp do kolejki dostępnych dla użytkownika powtórek (tych, których czas między powtórkami już upłynoł). Same UI setu definiuje jak zaprezentować użytkownikowi tą kolejkę.
- '/api/{set}/review/lastSession' - Daje informacje zwrotne o ostatnio zakończonej sesji - może zostać wykorzystny do wyświetlenia np. ekranu podsumowania co dany użytkownik powtórzył podczas danej sesji.
- '/api/{set}/review/progress' -

5. Definiowanie nowych zestawów

6. Podsumowanie

7. Możliwe sposoby rozwoju aplikacji